OmniDB: A Unified Cross-Platform Video Game Database for Discovery, Edition Comparison and Personal Game Spending

Balamurugan RMM and Vaishnavperumal M
Department of Computer Science and Design
Rajalakshmi Engineering College, Chennai, Tamil Nadu, India
balamurugan.rmm.2024.csd@rajalakshmi.edu.in | vaishnavperumal.m.2024.csd@rajalakshmi.edu.in
Under the guidance of Dr. P. Revathy

Abstract— Video game information is distributed across storefronts, console ecosystems, publishers and community databases. A player researching one title may need to compare editions, identify platform availability, distinguish remasters from original releases, and understand where a game can be purchased. This fragmentation becomes harder when the same game appears under different store listings and when platform-specific editions are represented inconsistently. This paper presents OmniDB, an existing student project developed in Google AI Studio, with a catalog of 325 games at the time of writing. The project vision is to provide one searchable record for a game while preserving its platform and edition-specific details across PC storefronts and console ecosystems. We describe the problem, data model, deduplication strategy, search and comparison workflow, proposed personal library and spending-tracker functions, and a practical architecture for moving from a frontend prototype toward a maintainable data-backed application. Unlike a benchmark study, this paper does not claim measured user or performance improvements: the current catalog size is the known project state, while backend services, automated price history and wallet functions are treated as design extensions until verified in the running implementation. The paper contributes a structured system design and an evaluation plan for a cross-platform game information platform.

Index Terms— video game database, cross-platform catalog, game discovery, data normalization, duplicate detection, storefront comparison, personal library, spending tracker.

I. INTRODUCTION

Digital games are distributed through a growing collection of storefronts and hardware ecosystems. A title may have separate listings on Steam, Epic Games Store and GOG, while console editions may be published for PlayStation, Xbox or Nintendo platforms. Store pages are useful for purchase and support, but they are not designed to act as one neutral index of a game across every platform. A player who wants to discover a title, compare availability, or identify the correct edition must often repeat the same search across multiple sites.

The problem is not simply the number of listings. Store listings can describe different editions of the same game, while similar names may refer to distinct releases. Remasters, definitive editions, regional variants, downloadable content and bundles require relationships that a flat list of titles cannot express reliably. A unified database therefore needs a canonical identity for each game and separate records for platform availability and store-specific offers.

OmniDB is a student project intended to address this fragmented discovery experience through the product vision: “One game. Every platform. One database.” The existing frontend was created in Google AI Studio and the current catalog contains 325 games. This paper documents the system concept and its technical direction without assuming that every proposed capability is already implemented.

A. Problem Statement

How can a single application organise video games from multiple platforms and storefronts into a consistent, searchable catalog while retaining differences between editions, stores and hardware? The system should reduce repeated searching, prevent duplicate catalog entries, present platform availability clearly, and provide a foundation for personal tracking without making unsupported claims about price or purchase data.

B. Objectives and Contributions

The work has five objectives: (1) define a cross-platform game record and platform-specific release model; (2) establish a duplicate-aware catalog workflow; (3) organise browsing through search, filters and comparable metadata; (4) specify an extensible backend and data-quality process; and (5) design optional library, wishlist and spending-tracker features. The main contribution is a practical architecture and data model suited to an existing prototype rather than a replacement project.

II. RELATED SYSTEMS AND DESIGN GAP

Digital storefronts such as Steam, GOG and the Epic Games Store provide authoritative information for the products sold through their own services [1]–[3]. Console platform holders similarly publish catalog and product information for their ecosystems [4]–[6]. These sources are essential for validating store-specific facts, but each is scoped to its own platform and interface. General game databases offer broad discovery, while price-comparison sites focus on current offers or historical prices. Their data models and coverage differ, so users may still need to reconcile information across several destinations.

OmniDB is positioned as a cross-platform discovery layer rather than a replacement storefront. It should link users to official product pages and identify the source and time associated with volatile facts such as price, availability and release date. A qualitative capability comparison is presented in Table I; it describes system categories, not a measured audit of named products.

A. Design Gap

The key design gap is the separation between game identity and a store listing. Treating every listing as a unique game inflates the catalog and makes comparisons unreliable. Conversely, merging records solely because their names are similar can combine unrelated releases. OmniDB therefore requires explicit relationships among a canonical game, editions, platforms and offers, with manual review for uncertain matches.

B. Scope

The known project state is a frontend prototype with 325 catalog entries. Cross-platform coverage is the intended scope and includes PC storefronts and console platforms. The precise completeness of each platform, live price integration, authentication, persistent user data and automatic synchronisation must be verified separately before being reported as implemented.

TABLE I
QUALITATIVE CAPABILITY COMPARISON BY SYSTEM CATEGORY

Capability	Single store	General game DB	Price tracker	OmniDB target
Cross-platform catalog	No	Often	Partial	Yes
Edition relationships	Store-scoped	Varies	Offer-focused	Explicit model
Store links	Yes	Sometimes	Yes	Yes
Personal library	Account-specific	Varies	Limited	Planned
Spending summary	Purchase history	Varies	Limited	Planned

Capability	Single store	General game DB	Price tracker	OmniDB target
Source / update time	Store-managed	Varies	Price-focused	Required for volatile data

III. SYSTEM REQUIREMENTS AND ARCHITECTURE

The architecture separates the user interface from catalog services, normalized game data, provider adapters and persistence. This separation allows the current frontend to remain intact while a backend is added incrementally. Figure 1 shows the logical layers. A production implementation should expose stable service interfaces so that mock data can be replaced with validated records without rewriting the presentation layer.

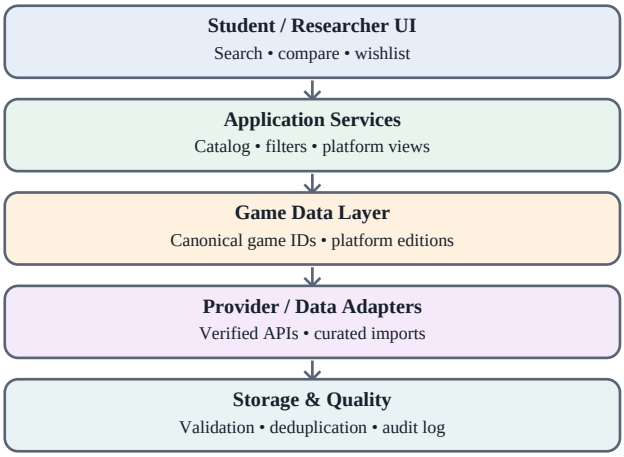


Fig. 1. Proposed layered architecture for OmniDB. The diagram is a target design; it is not a claim that every backend layer is already deployed.

A. User-facing requirements

The primary workflow is discovery: search by title, filter by genre or platform, open a game detail view, and compare available editions. Results should make platform availability and data freshness visible. Optional user-oriented modules include a library of owned games, a wishlist, and a wallet-style spending summary. These features should be introduced as separate modules so that the core catalog remains useful without sign-in or purchase-history data.

B. Quality requirements

Catalog correctness is more important than apparent size. Search should remain responsive as the catalog grows; duplicate detection should be explainable; external links should resolve to legitimate sources; and price values should never be shown without currency, source and update time. The system should also degrade gracefully when a provider is unavailable. These requirements favour a modular architecture, validated imports and a clear distinction between confirmed records and placeholders.

IV. DATA MODEL AND NORMALIZATION

The central data model separates the identity of a game from the ways it is released and sold. A canonical Game entity stores stable attributes such as title, franchise, developer, publisher, original release year, genres and a short description. An Edition entity captures release-specific attributes such as edition name, remaster status, included content and parent edition. A PlatformRelease entity associates an edition with a platform and may include release date, supported modes and platform-specific notes. A StoreOffer entity records the storefront, URL, current displayed price, currency, region and last-checked timestamp.

This structure avoids placing a single platform or store field directly on the canonical game. One game may have multiple platform releases, and one release may be represented by more than one offer. The same normalized design also supports future additions without changing the meaning of existing records.

TABLE II

CORE ENTITIES AND THEIR RESPONSIBILITIES

Entity	Representative fields	Purpose
Game	game_id, canonical_title, franchise, developer, genres	Stable identity and discovery metadata
Edition	edition_id, game_id, edition_name, edition_type	Distinguishes remakes, remasters and bundles
PlatformRelease	release_id, edition_id, platform, release_date	Records platform-specific availability
StoreOffer	offer_id, release_id, store, URL, price, currency, checked_at	Links an edition to a source listing
UserLibrary	user_id, game_id, status, acquired_on	Tracks owned, playing or completed games
WishlistItem	user_id, game_id, target_price, created_at	Stores a user's saved interest
SpendEntry	user_id, amount, currency, date, source	Supports user-entered spending summaries

A. Duplicate Detection

A safe deduplication process should use more than a title string. Candidate matches can be ranked using normalized title, developer or publisher, release year, franchise, edition type and platform. Exact identifiers from a trusted provider are stronger evidence than fuzzy title similarity. If two entries differ in release type or included content, they should remain separate editions connected to the same canonical game rather than being collapsed into one record.

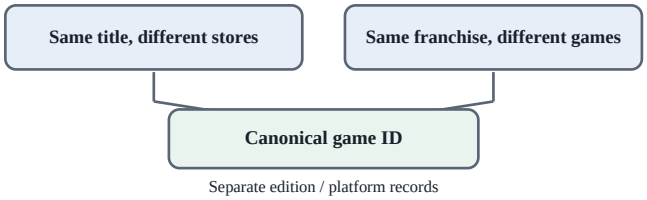


Fig. 2. Conceptual deduplication: related records resolve to one canonical identity while preserving separate editions and platform releases.

A practical workflow first normalizes case, punctuation and whitespace, then creates candidate pairs. High-confidence pairs can be linked automatically; medium-confidence pairs should enter a review queue; low-confidence pairs remain distinct. Every merge should be reversible and recorded in an audit log. This protects the existing 325-game catalog from destructive bulk edits.

B. Data Provenance

Every field should have a known provenance where feasible. Stable metadata may be curated from reliable catalog sources, while live prices and stock status require a timestamp and region. If a value is unavailable, the interface should display "unknown" rather than infer a value. User-entered spending entries must be kept separate from verified store prices, because a recorded purchase amount may include discounts, tax, bundles or currency conversion.

V. SEARCH, COMPARISON AND USER WORKFLOWS

The proposed workflow is illustrated in Fig. 3. The user begins with a query or browse filter, the catalog resolves a canonical game identity, and the detail page presents its editions and platform releases. Store offers are displayed as distinct records with direct source links. User actions such as saving to a wishlist or recording a purchase are handled independently of the public catalog.



Fig. 3. End-to-end catalog workflow from search to comparison and user tracking.

A. Search and Filtering

Search should support title matching and sensible partial queries. Filters may include platform, genre, release period, multiplayer support and availability. Sorting should be deterministic and explainable. If fuzzy matching is used, it should improve retrieval without silently merging records. A result card should communicate the canonical title and available platforms; a detail page should show edition-level distinctions.

B. Cross-platform Comparison

Comparison should align equivalent attributes and flag unavailable values. For example, the same title may have different release dates, editions, technical features or prices on different platforms. A platform comparison should not imply that the editions are identical. Prices should be compared only when currency and region are known, and conversions should identify the exchange rate and time if conversion is implemented.

C. Library, Wishlist and Spending Tracker

A personal library can record statuses such as owned, playing, completed or abandoned. A wishlist can store a desired game and optional target price. A spending tracker can total purchases entered by the user, group them by month or platform, and distinguish actual paid amounts from current listed prices. These are proposed extensions unless verified in the running prototype. The application should not claim access to private storefront purchase history without an explicit authorised integration.

VI. IMPLEMENTATION STRATEGY

OmniDB already has a frontend and should be extended rather than rebuilt. A staged implementation reduces risk: first define typed data contracts and a single catalog service; next add persistence and validation; then introduce provider adapters and user-specific modules. Mock data should be labelled clearly during development. When replacing it, the UI should continue to use the same data contract so that existing working components are preserved.

A. Backend and Persistence

A conventional web architecture could use a REST API with a relational database or a document database. The choice should follow the actual shape of the current codebase and deployment constraints rather than be made in isolation. Relational storage is well suited to explicit relationships among games, editions, platforms and offers; a document database can also work if relationships and uniqueness rules are enforced consistently. Node.js/Express and MongoDB are possible implementation choices, not verified facts about the existing prototype.

B. Data Ingestion and Validation

Provider adapters should be isolated behind interfaces. Use official APIs, permitted datasets or manually curated imports; do not depend on brittle scraping that violates source terms. An ingestion job should validate required fields, normalize platform names, check URL structure, identify duplicate candidates and store an import report. Updates to price-like fields should not overwrite provenance or historical values. Failures should be logged and retried with limits.

C. Security and Reliability

Public catalog reads can be separated from authenticated user writes. User library entries, wishlist items and spending records require ownership checks and should never be exposed through public catalog endpoints. Secrets belong in server-side environment configuration, not in frontend bundles. Rate limits, input validation, pagination, backup procedures and clear error states are necessary before public deployment.

VII. USER WORKFLOWS AND EDGE-CASE HANDLING

A cross-platform catalog must behave predictably when information is incomplete, conflicting or temporarily unavailable. This section defines representative user journeys and the rules that keep the interface consistent. These are proposed system behaviours to guide implementation and testing; they should not be interpreted as evidence that every workflow is already active in the current prototype.

A. Discovering a Game

A user may begin with a title, a genre, a franchise or a platform. Search results should return canonical game records rather than one result for every storefront listing. The result card should show the main title, a release or edition qualifier where needed, a small set of useful metadata and the platforms known to have a release. If a title has several editions, the user should be able to expand the record to inspect them rather than seeing multiple indistinguishable cards. When no exact match exists, the interface should offer near matches but label them as suggestions.

B. Comparing Editions and Platforms

A comparison view should use a common set of attributes and distinguish three states: confirmed value, unknown value and not applicable. An empty field must not be silently interpreted as a negative result. For example, missing multiplayer metadata does not prove that a game has no multiplayer mode. Edition names should remain visible throughout the comparison because a remaster or complete edition can contain different content from the original release. If two stores sell the same edition, they should appear as separate offers attached to the same release record.

C. Handling Conflicting or Stale Data

Provider data may disagree about release dates, titles or supported platforms. The application should retain the source value and record the time it was checked, then apply a documented precedence rule or route the disagreement for review. A newer timestamp alone is not always sufficient: an official platform listing may be more authoritative for its own store availability than a third-party summary. Volatile values should expire from a freshness perspective after a configurable interval. If a refresh fails, the previous value may remain visible with a stale-data indicator rather than being presented as live.

D. Accessibility and Interaction Design

The core search and comparison tasks should be possible using a keyboard, readable focus indicators and controls with meaningful labels. Platform icons should not be the only way to communicate platform names. Text and background contrast should remain sufficient in light and dark themes, and status messages should explain what happened after an action such as saving a game. Long catalogs require pagination or incremental loading, but the interface should preserve the active filters and search query when the user returns from a detail page.

TABLE IV

EDGE CASES AND EXPECTED SYSTEM RESPONSE

Case	Expected response	Reason
Same title, different release	Keep separate editions linked to one game where justified	Avoids false merges
Similar titles, unrelated games	Retain separate canonical IDs	Name similarity is insufficient
Price not refreshed	Show last checked time or mark unavailable	Avoids implying live accuracy
Provider unavailable	Use cached metadata with stale indicator	Graceful degradation
Unknown platform field	Show unknown, not “unsupported”	Distinguishes missing data from a negative fact

Case	Expected response	Reason
Duplicate import row	Reject or flag using uniqueness checks	Protects catalog integrity

VIII. DATA GOVERNANCE, SECURITY AND MAINTENANCE

A catalog becomes less trustworthy when records can be changed without traceability. OmniDB should treat data quality as an ongoing process rather than a one-time import task. Governance rules define who can edit records, how uncertain matches are reviewed, how provider terms are respected, and how corrections can be reversed. This is particularly important as the catalog grows beyond its current 325 entries and more sources are introduced.

A. Data Lifecycle

A record should pass through explicit states: imported, validated, published, stale, under review and retired. Importing a row does not automatically make it publication-ready. Validation checks required fields, acceptable platform identifiers, URL syntax, release-type consistency and references to existing canonical IDs. Records with unresolved conflicts can be kept in a review queue so they do not silently contaminate user-facing comparisons. A retired listing should generally preserve its historical identity while no longer appearing as an active offer.

B. Change History and Reversibility

Updates to a canonical title, edition relationship or platform mapping can affect search and user libraries. A change log should therefore capture the affected record, old and new values, source, timestamp and reason. High-impact operations such as merging two game IDs should be reversible. If user-specific records reference an ID that is merged, the migration must move those references safely and avoid duplicate wishlist or library entries. A periodic integrity job can detect orphaned references and inconsistent edition relationships.

C. Privacy and Access Control

The public catalog and private user data have different access requirements. Public read endpoints may expose approved game metadata, but a user's library, wishlist and spending entries should be accessible only to that user and authorised services. The application should collect the minimum personal information required for its features. Spending totals can be calculated from user-entered records without collecting payment credentials or attempting to infer a user's complete purchase history. Authentication, if introduced, must be paired with server-side ownership checks rather than relying only on hidden interface controls.

D. Threats and Defensive Controls

The main security risks include injection through search inputs, unauthorised changes to catalog records, exposure of API keys, insecure direct-object references in user endpoints, and excessive automated requests to third-party services. Input validation and parameterised database operations reduce injection risk. Secrets should remain on the server, write operations should be authenticated and authorised, and administrative actions should be logged. Rate limits and bounded retries help prevent accidental request storms. External URLs should be validated and rendered as links to the intended domain to reduce misleading or unsafe redirects.

E. Backup, Recovery and Operations

A persistent backend needs a defined recovery process. Backups should be scheduled according to the acceptable data-loss window and periodically tested by restoring into a separate environment. Database migrations should be versioned and tested against a copy of the data before deployment. Operational monitoring should distinguish application errors, provider failures and data-validation failures. A simple health endpoint can report whether the API and primary database are reachable without exposing credentials or private information.

TABLE V

DATA QUALITY CHECKS BEFORE PUBLICATION

Check	Validation rule	Failure handling
Identity	Canonical ID is unique and stable	Reject duplicate key; create review item
Relationships	Edition and release references exist	Block publication until repaired
Source URL	Valid scheme and permitted domain	Flag for correction
Volatile fields	Price includes currency and checked time	Mark incomplete or stale
Platform value	Matches controlled platform vocabulary	Normalize or queue for review
Change history	Editor, timestamp and reason recorded	Reject unaudited high-impact change

IX. DISCUSSION AND DESIGN TRADE-OFFS

The architecture proposed for OmniDB involves trade-offs between coverage, correctness, development effort and freshness. A system that imports large quantities of unverified data may look comprehensive while producing unreliable platform comparisons. A smaller, validated catalog can be more useful because users can understand where each value came from and whether it is current. The current 325-entry catalog is therefore a starting point for establishing quality controls, not proof that every game or platform has been covered.

A. Breadth Versus Confidence

Adding every possible platform and store at once would increase the number of source adapters and the number of cases that must be tested. A staged rollout should first establish a reliable common model and validate a limited group of sources. Additional platforms can then be added using the same contract and quality checks. Coverage reporting should distinguish sources that have been integrated, sources that have been manually curated, and sources that are only planned. This avoids presenting intended scope as completed coverage.

B. Automation Versus Human Review

Automated matching can process large imports quickly, but ambiguous titles and editions need human judgement. A review queue is a deliberate quality feature, not a failure of automation. The interface for reviewers should show the evidence behind a candidate match—title, developer, year, platform and source identifiers—and allow the reviewer to merge, keep separate or defer. Feedback from reviewed cases can improve matching rules, but automatic decisions should remain auditable and reversible.

C. Current Price Versus Purchase Value

The project's proposed spending feature creates a useful distinction between market information and personal financial records. Current store price answers what a user might pay now; a spending entry answers what that user reports having paid at a particular time. These values should not be combined into a single "game value" field. Refunds, bundles, gifts, subscription access and multiple currencies complicate totals, so the first version should clearly state which transactions are included and allow users to correct their own entries.

D. Responsible Use of External Data

External catalog data should be used according to the source's terms, attribution requirements and technical limits. Where an official API exists, its documented authentication and rate limits should be followed. If data is manually curated, the project should keep source links and review dates. The system should not imply an official partnership with any storefront unless one exists. Trademarks and game artwork should be used in ways consistent with the applicable permissions and platform policies; this paper does not assert that all assets have been cleared for redistribution.

E. Practical Priorities

For a student team, the highest-value order of work is to protect the existing frontend, normalise game identity, validate platform relationships and make source freshness visible. These steps improve the reliability of the core catalog and provide a stable foundation for later user features. Automatic price history, notifications and broad provider integrations should follow only when data access, persistence and operational monitoring can support them. This ordering reduces the risk of spending effort on advanced features while basic records remain inconsistent.

X. LIMITATIONS AND FUTURE WORK

The current verified project fact is a 325-game catalog. No controlled performance benchmark, user study or independently verified accuracy result is available in the project context used for this paper. Evaluation should therefore be treated as future work, not as a set of achieved results. A repeatable plan can still define the measurements needed to decide whether the system is useful and reliable.

A. Catalog Quality Metrics

Catalog quality should be assessed using duplicate rate, missing-field rate, broken-link rate, platform-coverage ratio and the proportion of volatile fields with valid timestamps. A manually reviewed sample should establish precision and recall for duplicate detection. Coverage must be reported with a defined denominator—for example, a chosen list of games and platforms—rather than by raw record count alone.

B. Performance and Usability

Search latency should be measured at increasing catalog sizes and under concurrent requests. A usability study should ask participants to locate a game, compare two editions and determine availability on a requested platform. Completion time, task success, error rate and a standard usability questionnaire can be recorded. Results should include sample size, test environment and uncertainty; until then, no numeric performance benefit is claimed.

TABLE III

PROPOSED EVALUATION METRICS

Dimension	Metric	Interpretation
Catalog integrity	Duplicate precision / recall	Quality of identity matching
Completeness	Required-field coverage	How many records are usable
Link reliability	Valid-source-link rate	Share of links resolving correctly
Search	Median and 95th percentile latency	Responsiveness under defined load
Usability	Task success and completion time	Ability to complete common tasks
Spending feature	Reconciliation error against entered data	Correctness of user-side totals

C. Test Dataset Construction

A useful test corpus should be designed to expose the difficult cases rather than contain only popular, unambiguous titles. It should include original releases, remakes, remasters, complete editions, bundles, games with identical names from different developers, and platform-exclusive releases. Each test record should have a manually checked reference label for canonical game identity, edition identity and platform availability. The corpus should include missing metadata and stale links so that the test measures how the system behaves when source data is imperfect.

D. Measurement Definitions

For duplicate detection, precision is TP/(TP+FP), where TP denotes correctly identified duplicate pairs and FP denotes pairs incorrectly merged. Recall is TP/(TP+FN), where FN denotes true duplicate pairs that were missed. Precision is especially important because a false merge can corrupt several related records and user lists; recall matters because missed duplicates make the catalog harder to browse. Both should be reported rather than collapsing quality into a single accuracy number.

E. User Study Protocol

A small formative study can recruit participants with different levels of gaming knowledge. Each participant should complete tasks such as finding a game on a specified platform, identifying the difference between two editions, and locating the source page for a listing. Task order should be varied to reduce learning effects. Observers should record task completion, time, incorrect selections and points at which the participant hesitates. A post-task questionnaire can ask whether platform availability, edition differences and data freshness were understandable.

F. Acceptance Criteria

Before a public release, the team should define minimum acceptance criteria for the core paths. Examples include no known broken links in the release test set, no unauthorised access to another user's library, no duplicate creation when the same import is retried, and no loss of existing catalog entries during a migration. Numerical thresholds for latency and completeness should be chosen after a baseline run on the actual hosting environment rather than invented in advance. A release checklist should preserve evidence of each test result.

G. Implementation Roadmap

The implementation can be organised into small, reversible milestones. Milestone 1 audits the current frontend and documents every working screen, component and data shape. Milestone 2 introduces a typed canonical catalog schema and a validation layer while keeping the current interface operational. Milestone 3 adds persistent storage and a read API behind the existing data-access boundary. Milestone 4 introduces import validation, source provenance and duplicate review. Milestone 5 adds authenticated personal library, wishlist and spending entries only after access controls are tested. Milestone 6 measures data quality and search performance using the protocol above.

H. Reproducibility and Reporting

A credible technical report should preserve the dataset snapshot date, schema version, import rules, test corpus, software versions and the exact commands or steps used to run the evaluation. If external source data cannot be redistributed, the project can publish record identifiers, transformation rules and aggregate counts without republishing restricted data. Every reported result should identify whether it comes from the live prototype, a manual audit, a controlled test or a simulation. This distinction prevents design goals from being mistaken for observed outcomes.

C. Baselines

Useful baselines include searching each storefront separately, comparing records without canonical identities, and searching the normalized OmniDB catalog. The test set should contain same-title editions, remasters, platform-specific releases and near-duplicate names. This makes it possible to measure whether normalization improves task completion without hiding important edition differences.

X. LIMITATIONS AND FUTURE WORK

The catalog's current size does not establish completeness or freshness. A list of 325 games can demonstrate the interface and initial data structure, but it cannot prove broad coverage of every platform or store. Provider terms and API availability may constrain automated updates. Prices vary by region and time, and a game's availability may differ by edition. These facts require source-aware data handling rather than a single universal price field.

Future work should prioritise: (1) audit the existing frontend and preserve functioning features; (2) formalise the canonical game, edition and platform-release schema; (3) add persistence and validation; (4) create a permitted data-import pipeline; (5) build duplicate review tools; (6) add user accounts only when required; (7) implement library, wishlist and spending summaries; and (8) conduct the catalog-quality, performance and usability evaluation described above. Price history and automatic price alerts should follow only after reliable timestamped data collection exists.

XI. CONCLUSION

This paper presented OmniDB, a cross-platform video game catalog project whose current known state is a frontend developed in Google AI Studio with 325 games. The paper defined the problem of fragmented storefront information and proposed a normalized model that separates canonical games, editions, platform releases and store offers. It also described search, comparison, duplicate detection, provenance, a modular backend strategy and an evaluation plan. Personal library, wishlist and spending-tracker capabilities are described as extensions rather than assumed completed features. The next step is not a wholesale rebuild: it is to preserve the existing interface, strengthen data identity and provenance, introduce validated persistence, and measure catalog quality before making claims about production readiness.

ACKNOWLEDGMENT

The authors thank Dr. P. Revathy for her guidance and the Department of Computer Science and Design, Rajalakshmi Engineering College, Chennai, Tamil Nadu, India, for support.

REFERENCES

[1] Valve Corporation, "Steam Store," official storefront. [Online]. Available: https://store.steampowered.com/

[2] Epic Games, "Epic Games Store," official storefront. [Online]. Available: https://store.epicgames.com/

[3] GOG, "GOG.com," official storefront. [Online]. Available: https://www.gog.com/

[4] Sony Interactive Entertainment, "PlayStation Store," official storefront. [Online]. Available: https://store.playstation.com/

[5] Microsoft, "Xbox," official gaming platform and store. [Online]. Available: https://www.xbox.com/

[6] Nintendo, "Nintendo," official gaming platform and store. [Online]. Available: https://www.nintendo.com/

[7] MongoDB, "MongoDB Manual," database documentation. [Online]. Available: https://www.mongodb.com/docs/

[8] Node.js, "Node.js Documentation." [Online]. Available: https://nodejs.org/docs/

[9] OWASP Foundation, "OWASP Top 10: Web Application Security Risks." [Online]. Available: https://owasp.org/www-project-top-ten/

[10] ISO/IEC 25010:2023, "Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model," International Organization for Standardization, 2023.